

SPRINT 4

Patricio – tu amigo robot

Presentado por:

Mari Dapcheva
Adenor Buret
Pablo Meana
Cesar Herrero
Santiago Aguirre
Juan Bautista Peris



Versión	Autor	Descripción	Fecha
v1.0	Pablo Meana	Versión inicial del documento. Creación de la estructura general, redacción de la concepción del producto y descripción de los paquetes ROS 2 del sistema.	30/03/2026
v1.1	Mari Dapcheva	Ampliación del documento con la descripción de la arquitectura del sistema, la interfaz web, la base de datos, la API interna, el plan de verificación de funcionalidades y el apartado de troubleshooting.	20/04/2026
v1.2	Juan Bta. Peris	Incorporación de la base teórica del proyecto: introducción con el estado del arte de robots educativos referentes, contexto del proyecto y	7/05/2026



		definición de los objetivos SMART de los cuatro pilares (Navegación Nav2, Interfaz Web, OpenCV y Reconocimiento IA).	
v1.3	Mari Dapcheva y Adenor Buret	Añadir apartado Persistencia de datos; Actualizar lógica de juegos; Actualización de índices y referencias; Corrección de numeración de apartados; Reorganización de figuras y tablas; Asignación de títulos y descripciones coherentes a todas las figuras y tablas; Homogeneización del estilo visual del documento; Creación de Índice de figuras y tablas automático; Referencia en el	2/06/2026



		texto de cada figura; Documentación de detección de caídas; Documentación de OpenCV; Documentación de módulos de IA;	
v1.4	Mari Dapcheva Juan Bta. Peris	Reorganización de la memoria y adaptación a la estructura requerida por la asignatura.	11/06/2026

Enlace github: <https://github.com/CJMIDNIGHT/patricio.git>

Enlace Trello: <https://trello.com/b/GA5La54N/robotica-patricio>



Índice:

1. Introducción	7
1.1 Descripción	7
1.2 Estado del arte y vigilancia tecnológica	7
2. Contexto del proyecto	10
3. Objetivos del proyecto	11
3.1 Objetivo general	11
3.2 Objetivos específicos	11
3.3 Objetivos SMART	12
4. Definición de requisitos del proyecto	13
5. Diseño	15
5.1 Arquitectura completa	15
5.1.1 Capa de usuario	16
5.1.2 Capa de comunicación	16
5.1.3 Capa del robot	17
5.2 Diseño del front-end	20
5.2.1 Landing page	21
5.2.2 Pantalla de registro	23
5.2.3 Pantalla de Inicio de sesión	24
5.2.4 Pantalla de Panel de control	25
5.2.5 Pantalla de Sección de Partidos	26
5.2.6 Pantalla de Sección de Notificaciones	27
5.2.7 Pantalla de la Sección de Perfil	27
5.3 Diseño del back-end	28
5.3.1 Base de datos	29
5.3.2 API Flask	30
5.3.3 Sistema de visión artificial	32
5.3.4 Inteligencia artificial y reconocimiento	35
5.3.5 Sistema de emociones	37
5.4 Explicación de las tecnologías necesarias	40



6. Implementación	41
6.1 Estructura de carpetas del proyecto	41
6.2 Implementación de los paquetes	42
6.2.1 Simulación del entorno	42
6.2.2 Navegación autónoma	43
6.2.3 Captación y procesamiento de imagen	43
6.2.4 Interfaz web	44
6.2.5 Juegos educativos	44
6.2.6 Persistencia de datos	45
6.3 Uso de OpenCV	45
6.3.1 Captación y preprocesado de imágenes	45
6.3.2 Detección corporal mediante MediaPipe	47
6.3.3 Integración con los juegos	47
7. Verificación de las funcionalidades implementadas	48
7.1 Metodología de pruebas	48
7.2 Pruebas unitarias	49
7.3 Pruebas a nivel de nodo	49
7.4 Pruebas integrales	49
7.5 Resultados obtenidos	49
8. Reflexión crítica del funcionamiento del grupo de trabajo y su evolución	50
9. Conclusiones y propuestas de mejora	53
9.1 Conclusiones	53
9.2 Propuestas de mejora	54



Índice de tablas y figuras:

Tabla 1. Comparación de funcionalidades entre los robots educativos analizados y la propuesta Patricio.	10
Tabla 2. Definición de los objetivos SMART del proyecto Patricio.	13
Figura 1. Arquitectura general del sistema Patricio	15
Figura 2. Página principal (landing page) de la plataforma web Patricio (1)	21
Figura 3. Página principal (landing page) de la plataforma web Patricio (2)	22
Figura 4. Página principal (landing page) de la plataforma web Patricio (3)	23
Figura 5. Página principal (landing page) de la plataforma web Patricio (4)	23
Figura 6. Pantalla de registro de nuevo usuario en la interfaz web de Patricio.	24
Figura 7. Pantalla de login en la interfaz web de Patricio.	25
Figura 8. Vista general del panel de control para tutor/empleado.	25
Figura 9. Sección de gestión y listado de partidas en el panel de control.	26
Figura 10. Sección de notificaciones e incidencias en la interfaz web.	27
Figura 11. Vista del perfil de usuario dentro de la plataforma Patricio.	27
Figura 12. Edición de datos personales y configuración del perfil de usuario.	28
Figura 13. Diagrama Entidad-Relación de la base de datos Patricio	29
Figura 14. Flujo de comunicación entre la interfaz web, la API Flask, la base de datos MySQL y ROS 2.	31
Figura 15. Arquitectura completa de la lógica de visión artificial.	33
Figura 16. Capa 1 de la lógica de visión artificial.	33
Figura 17. Capa 2 de la lógica de visión artificial.	34
Figura 18. Capa 3 de la lógica de visión artificial.	35
Figura 19. Pipeline completo del sistema de voz e inteligencia artificial.	36
Figura 20. Representación de la emoción Feliz en la interfaz del robot Patricio.	38
Figura 21. Representación de la emoción Triste en la interfaz del robot Patricio.	38
Figura 22. Representación de la emoción En alerta en la interfaz del robot Patricio.	39
Figura 23. Representación de la emoción Concentrado en la interfaz del robot Patricio.	39
Tabla 3. Tecnologías utilizadas en el desarrollo del proyecto Patricio y su función dentro del sistema.	41
Figura 24. Visualización en tiempo real de la cámara del robot desde la interfaz web.	46



1. Introducción

1.1 Descripción

Durante la fase de concepción del producto, realizamos un análisis exhaustivo del mercado y las necesidades de los usuarios con el objetivo de desarrollar una solución innovadora en el ámbito de la educación infantil.

A través de vigilancia tecnológica, ideación colaborativa y validación de usuario, identificamos una oportunidad clara: la falta de atracción al aprendizaje.

Así nació la idea de Patricio, como un robot educativo orientado a niños de 4 a 6 años, con el objetivo de fomentar el aprendizaje mediante interacción lúdica e integración de inteligencia artificial para una experiencia autónoma y adaptativa.

La propuesta se apoya en un público objetivo claro (educación infantil temprana) con unas funcionalidades previstas descritas en el README del proyecto: matemáticas y alfabeto, narración de cuentos y chistes, juegos de movimiento ("pilla pilla", "el pollito inglés"), expresión en pantalla, navegación autónoma y reconocimiento de voz.

1.2 Estado del arte y vigilancia tecnológica

Durante la fase inicial del proyecto se realizó un ejercicio de vigilancia tecnológica para identificar las soluciones existentes en el ámbito de la robótica educativa orientada a edades tempranas. El análisis se centró en aquellas plataformas que comparten parte del público objetivo o de las funcionalidades de Patricio, con el fin de detectar el hueco de mercado que justifica el proyecto y de establecer la propuesta de valor diferencial. Se han seleccionado tres referentes representativos del estado actual del sector, cada uno cubriendo un nicho distinto del problema.

Bee-Bot (TTS Group)



Bee-Bot es uno de los robots educativos más extendidos en las aulas de educación infantil y primer ciclo de primaria en España. Diseñado con forma de abeja y orientado a niños a partir de tres años, su finalidad es introducir los conceptos básicos de programación, pensamiento computacional y razonamiento secuencial. Su funcionamiento se basa en una botonera de cuatro direcciones situada en el dorso del robot, mediante la cual el niño define una secuencia de hasta doscientos movimientos que el dispositivo ejecuta sobre una cuadrícula impresa en una alfombrilla temática.

Aunque Bee-Bot cumple con solvencia su misión didáctica en lo relativo a lateralidad, ubicación espacial y resolución de problemas, presenta limitaciones evidentes para un proyecto más ambicioso: carece de navegación autónoma, no incorpora inteligencia artificial ni capacidades de reconocimiento, no permite interacción verbal con el niño y no ofrece ninguna interfaz para el seguimiento por parte del tutor o el educador.

Miko 3 (Emotix)

Miko 3 representa el extremo opuesto del espectro: un robot sobremesa con pantalla expresiva orientado a niños de cinco a diez años, que apuesta de forma decidida por la inteligencia artificial conversacional. Incorpora procesamiento de lenguaje natural, reconocimiento de voz y reconocimiento facial, una cámara gran angular y un motor de inteligencia emocional que identifica y recuerda los estados de ánimo del niño. Su catálogo de contenidos incluye juegos educativos, historias, programación visual y aplicaciones de marcas como Disney.

Sus principales limitaciones desde la perspectiva del proyecto son tres: se trata de una plataforma cerrada y propietaria que no permite extensión por parte del docente; el robot es estático y no se desplaza por el entorno del niño, por lo que la interacción es siempre sedentaria y centrada en una pantalla; y la franja de edad recomendada queda por encima del público objetivo de Patricio.

Moxie (Embodied)

Moxie, desarrollado por la empresa Embodied, es un robot social diseñado para apoyar el aprendizaje socioemocional de niños de cinco a diez años a través de



misiones semanales sobre amistad, empatía o autorregulación emocional. Combina reconocimiento facial y de voz, una pantalla con expresiones faciales animadas y un currículo elaborado por especialistas en desarrollo infantil. Al igual que Miko, comparte tres limitaciones que abren un espacio para una propuesta alternativa: el ecosistema es propietario, el robot es estático y el rango de edad recomendado deja fuera la franja de cuatro a seis años en la que se concentran las necesidades educativas que motivan este proyecto.

Propuesta de valor de Patricio

El análisis comparado revela una oportunidad clara en el mercado: no existe actualmente una solución de robótica educativa orientada al rango de cuatro a seis años que combine, de forma simultánea, inteligencia artificial conversacional, reconocimiento por visión por computador, navegación autónoma por el entorno, juegos físicos que impliquen movimiento corporal del niño y una interfaz web abierta para el seguimiento por parte del tutor o el educador. Bee-Bot resuelve la programación elemental pero carece de IA y de navegación; Miko y Moxie resuelven la IA conversacional pero permanecen estáticos y se dirigen a una franja de edad superior. Patricio se sitúa precisamente en esa intersección no cubierta, apoyándose además en una pila tecnológica abierta basada en ROS 2 y Nav2 que garantiza la extensibilidad y la reutilización académica del trabajo realizado.

La siguiente tabla resume la comparativa entre los tres referentes y la propuesta de Patricio:

Característica	Bee-Bot	Miko 3	Moxie	Patricio
Rango edad	3-6 años	5-10 años	5-10 años	4-6 años
Navegación Autónoma	No	No (Estático)	No (Estático)	Si (Nav2)



Reconocimiento por IA	No	Voz y Facial	Voz y Facial	Voz y visión (OpenCV)
Juegos Físicos en el espacio	No	No	No	Sí (pilla-pilla, escondite, etc ...)
Interfaz para tutor/educador	No	App Móvil	App Móvil	Web multirol
Plataforma	Firmware cerrado	SO Propietario	SocialX (propietario)	ROS 2 + Gazebo (abierto)

Tabla 1. Comparación de funcionalidades entre los robots educativos analizados y la propuesta Patricio.

2. Contexto del proyecto

En los últimos años la tecnología ha adquirido un papel cada vez más importante en el aprendizaje y entretenimiento infantil. Sin embargo, muchos de los dispositivos utilizados por los niños se centran únicamente en contenidos digitales y ofrecen una interacción limitada con el entorno físico.

Además, debido al ritmo de vida actual, no siempre es posible que los padres o tutores dediquen todo el tiempo que les gustaría a realizar actividades educativas o de ocio con sus hijos. Esto ha impulsado el desarrollo de nuevas herramientas tecnológicas capaces de complementar estas actividades y ofrecer experiencias más dinámicas e interactivas.

En este contexto surge Patricio, un robot educativo diseñado para niños de entre 4 y 6 años. El objetivo del proyecto es proporcionar una plataforma capaz de combinar aprendizaje, entretenimiento e interacción social mediante diferentes actividades, juegos y sistemas de comunicación adaptados al público infantil.



Para ello, el robot incorpora funcionalidades como navegación autónoma, visión artificial, reconocimiento por voz, expresiones faciales e interfaz web de supervisión. Estas características permiten crear una experiencia más cercana y participativa, favoreciendo la interacción entre el niño y el robot y ofreciendo al mismo tiempo herramientas de seguimiento para padres y tutores.

El proyecto se ha desarrollado utilizando tecnologías abiertas como ROS 2, OpenCV, Flask y MySQL, con el objetivo de crear una solución flexible, ampliable y accesible para futuras mejoras.

3. Objetivos del proyecto

3.1 Objetivo general

El objetivo general del proyecto es desarrollar un robot educativo denominado Patricio, orientado a niños de entre 4 y 6 años, capaz de combinar navegación autónoma, visión artificial, reconocimiento e interacción mediante una interfaz web. El sistema debe proporcionar actividades educativas y de entretenimiento que favorezcan la participación del usuario, apoyándose en tecnologías abiertas basadas en ROS 2 y herramientas de inteligencia artificial.

3.2 Objetivos específicos

Para alcanzar el objetivo general se han definido los siguientes objetivos específicos:

- Implementar un sistema de navegación autónoma basado en Nav2 que permita al robot desplazarse de forma segura por el entorno evitando obstáculos.
- Desarrollar una interfaz web que permita controlar el robot y supervisar su estado de forma remota desde un navegador.



- Integrar técnicas de visión artificial mediante OpenCV para procesar la información capturada por la cámara y utilizarla en diferentes funcionalidades del sistema.
- Incorporar capacidades de reconocimiento basadas en inteligencia artificial que permitan identificar comandos de voz y, cuando sea posible, reconocer personas u objetos.
- Desarrollar actividades y juegos educativos que permitan aprovechar las capacidades de navegación, visión e interacción implementadas.

3.3 Objetivos SMART

Los objetivos específicos definidos para el proyecto se han estructurado siguiendo la metodología SMART (Specific, Measurable, Achievable, Relevant and Time-bound), con el fin de garantizar que sean claros, medibles y verificables.

Pilar	Específico (S)	Mediable (M)	Alcanzable (A)	Relevante (R)	Temporal (T)
Navegación Nav2	Desplazamiento autónomo en el entorno simulado evitando obstáculos.	Tasa de éxito $\geq 90\%$ en pruebas de navegación punto a punto.	Stack Nav2 sobre ROS 2 Jazzy ya validado en TurtleBot3.	Funcionalidad básica exigida por la asignatura y soporte del resto de pilares.	Sprint 2 (verificado en Sprint 3).



Interfaz Web	Control remoto y monitorización en tiempo real desde el navegador.	10/10 pruebas funcionales del plan de validación superadas.	Frontend HTML/JS + Flask + rosbridge (puerto 9090).	Permite operar el robot sin conocimientos de ROS.	Sprint 3 (entregable actual).
OpenCV	Procesamiento de imagen de la cámara para apoyo a la interacción.	≥ 1 pipeline OpenCV integrado y verificable desde la interfaz web.	Cámara del TurtleBot3 ya publicada en /camera/image.	Habilita la visión por computador que diferencia a Patricio de sus referentes.	Sprints 3-4.
Reconocimiento (IA)	Reconocimiento de voz y objetos/personas.	≥ 80 % de aciertos en el dataset de validación construido por el equipo.	Combinación de modelos (voz y modelo de visión sobre OpenCV).	Núcleo de la propuesta de valor educativa y adaptativa.	Sprint 3-4 (integración con Nav2 y web).

Tabla 2. Definición de los objetivos SMART del proyecto Patricio.

4. Definición de requisitos del proyecto

A partir de las necesidades identificadas durante la fase de análisis y de las historias de usuario definidas por el equipo, se establecieron los siguientes requisitos funcionales para el sistema:

- Mostrar emociones y expresiones faciales mediante una pantalla integrada en el robot.
- Disponer de un entorno de simulación que permita validar las funcionalidades antes de su despliegue en el robot real.



- Implementar actividades educativas relacionadas con el aprendizaje de matemáticas básicas.
- Incorporar actividades para el aprendizaje del alfabeto.
- Permitir la narración de cuentos educativos de forma interactiva.
- Implementar el juego de Pilla-Pilla utilizando navegación autónoma y visión artificial.
- Implementar el juego del Escondite mediante navegación autónoma y detección del usuario.
- Implementar el juego del Calamar utilizando técnicas de visión artificial para la detección de movimiento.
- Permitir la interacción mediante comandos de voz.
- Incorporar funcionalidades de visión artificial basadas en OpenCV para el procesamiento de imágenes.
- Detectar movimientos corporales mediante tecnologías de visión artificial como MediaPipe.
- Detectar posibles caídas del usuario y generar alertas cuando sea necesario.
- Proporcionar una interfaz web para el control y supervisión remota del robot.
- Mostrar el estado y las emociones del robot en tiempo real a través de la interfaz web.
- Almacenar información de usuarios, partidas e incidencias en una base de datos para garantizar la persistencia de los datos.

Además, se definieron varios requisitos no funcionales relacionados con la arquitectura y el funcionamiento del sistema:

- Utilizar ROS 2 Jazzy como plataforma principal para el desarrollo del robot.
- Garantizar la comunicación entre la interfaz web y ROS 2 mediante Rosbridge y WebSockets.
- Permitir la ejecución del sistema tanto en simulación como en el robot físico.
- Utilizar MySQL como sistema de persistencia de datos.



- Diseñar una arquitectura modular que facilite el mantenimiento y la incorporación de nuevas funcionalidades.

5. Diseño

5.1 Arquitectura completa

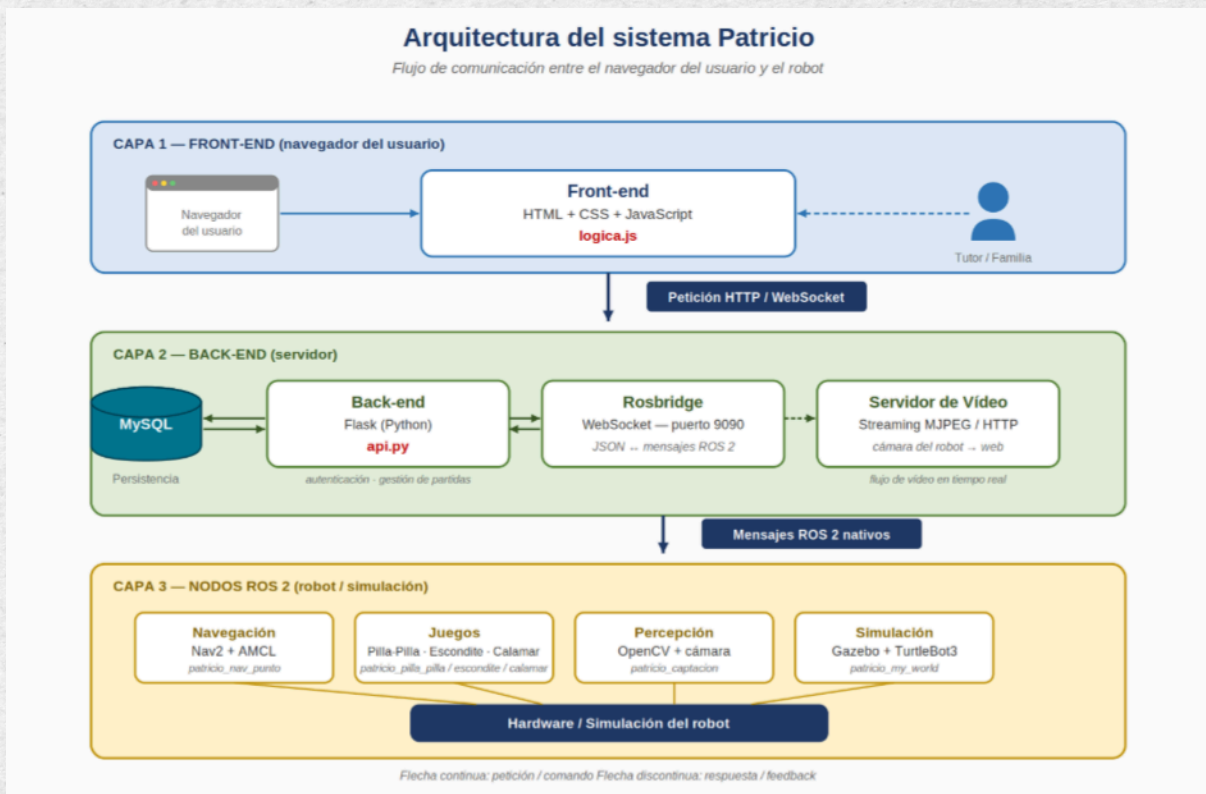


Figura 1. Arquitectura general del sistema Patricio

El diagrama que aparece en la figura 1 representa el flujo completo de datos del proyecto Patricio, desde que un operador pulsa un botón en el navegador hasta que el robot ejecuta una acción en el simulador. La arquitectura está dividida en tres capas bien diferenciadas, cada una con una responsabilidad clara y unos componentes específicos

5.1.1 Capa de usuario

Esta capa engloba todos los elementos con los que interactúa directamente el usuario. La interfaz web es servida por el paquete `patricio_web` y puede accederse desde un navegador mediante la dirección IP del robot.

El sistema está orientado a padres, madres o tutores, que pueden registrarse e iniciar sesión para acceder a las diferentes funcionalidades de la plataforma. Una vez autenticados, los usuarios pueden consultar información sobre las actividades realizadas, gestionar su perfil y supervisar la interacción entre el niño y el robot.

Además, existen componentes encargados de proporcionar información en tiempo real sobre el estado del sistema. Entre ellos destaca `script_monitor.js`, responsable de monitorizar diferentes parámetros del robot, y `draw_occupancy_grid.js`, que permite representar el mapa de ocupación y la posición del robot directamente en el navegador.

La comunicación con ROS 2 desde la interfaz web se realiza mediante la librería `roslibjs`, que permite publicar y suscribirse a tópicos ROS sin necesidad de que el usuario tenga ROS instalado localmente. Gracias a esta solución, el acceso al sistema resulta sencillo y transparente para usuarios sin conocimientos técnicos.

Comunicación mediante WebSocket

La conexión entre la interfaz web y ROS 2 utiliza el protocolo WebSocket a través del puerto 9090, proporcionado por Rosbridge.

A diferencia de HTTP tradicional, WebSocket mantiene un canal de comunicación bidireccional permanente, permitiendo:

- Enviar comandos desde la web al robot.
- Recibir información del estado del robot en tiempo real.
- Actualizar continuamente la interfaz sin necesidad de recargar la página.

5.1.2 Capa de comunicación



La capa de comunicación está formada principalmente por el nodo `rosbridge_server`, encargado de actuar como puente entre la interfaz web y el ecosistema ROS 2.

Su función consiste en:

- Recibir mensajes JSON enviados desde el navegador.
- Convertir dichos mensajes a tipos nativos de ROS 2.
- Publicarlos en los tópicos, servicios o acciones correspondientes.
- Enviar de vuelta al navegador la información generada por los distintos nodos del robot.

Gracias a este diseño, el front-end y el sistema ROS 2 quedan completamente desacoplados, facilitando el mantenimiento y la ampliación del proyecto.

La comunicación interna utiliza los tres mecanismos principales de ROS 2:

- **Topics**, para información continua como posición, velocidad o sensores.
- **Services**, para operaciones puntuales que requieren respuesta.
- **Actions**, para tareas largas que permiten seguimiento y cancelación.

5.1.3 Capa del robot

Esta capa contiene toda la lógica funcional del sistema y está formada por distintos módulos especializados.

Navegación autónoma

El paquete `patricio_nav_punto` implementa la navegación autónoma del robot mediante Nav2.

Entre sus responsabilidades destacan:

- Gestión de mapas.
- Localización mediante AMCL.
- Planificación de rutas.



- Control del movimiento.

Los principales tópicos utilizados son;

- /cmd_vel
- /odom
- /map

Además, la navegación se realiza mediante la acción:

- /navigate_to_pose

Simulación

El paquete patricio_my_world contiene el entorno de simulación desarrollado en Gazebo.

Este módulo incluye:

- El mundo virtual utilizado durante las pruebas.
- Los modelos tridimensionales del entorno.
- El modelo del TurtleBot3.
- La representación facial del robot.

Captación y procesamiento de imagen

El paquete patricio_captacion constituye la primera etapa del sistema de visión.

Sus funciones principales son:

- Capturar imágenes desde la cámara física.
- Gestionar automáticamente el cambio entre cámara real y simulada.
- Aplicar un preprocesado básico a las imágenes.
- Publicar una única fuente de imagen para el resto de módulos.

Visión artificial

El paquete patricio_vision implementa el procesamiento avanzado de imagen mediante MediaPipe y OpenCV.



Entre sus funciones destacan:

- Detección de personas.
- Seguimiento corporal.
- Obtención de landmarks corporales.
- Cálculo de movimiento y posición relativa.

Persistencia de datos

El paquete `patricio_db_interface` proporciona acceso a la base de datos desde ROS 2.

Para ello expone servicios que permiten:

- Registrar partidas finalizadas.
- Registrar incidencias detectadas durante la ejecución.

Internamente, estos servicios se comunican con la API Flask responsable del acceso a MySQL.

Juegos educativos

El sistema incorpora varios módulos específicos para cada actividad:

- `patricio_pilla_pilla`
- `patricio_escondite`
- `patricio_calamar`

Cada uno de ellos implementa la lógica correspondiente y recibe las órdenes de inicio mediante servicios ROS personalizados definidos en el paquete `patricio_interfaces`.

Interfaces personalizadas

El paquete `patricio_interfaces` define los tipos de mensajes, servicios y acciones personalizados utilizados por el sistema.



Entre ellos destaca el servicio StartGame.srv, encargado de iniciar las distintas actividades disponibles.

Motor de simulación

La base del sistema está formada por Gazebo, encargado de simular:

- La física del entorno.
- Los sensores virtuales.
- El movimiento del robot.
- La interacción con los distintos objetos del escenario.

Flujo general de comunicación

El usuario interactúa con la interfaz web mediante el navegador. Los comandos generados son enviados a través de Rosbridge utilizando WebSocket. Una vez convertidos a mensajes ROS 2, son procesados por los distintos módulos del robot.

Dependiendo de la acción solicitada, el sistema puede iniciar una navegación autónoma, ejecutar una actividad educativa o procesar información procedente de los sensores. Finalmente, el estado actualizado del robot es enviado de nuevo al navegador, donde se representa en tiempo real mediante los componentes de monitorización y visualización del sistema.

5.2 Diseño del front-end

La interfaz web de Patricio ha sido diseñada con el objetivo de proporcionar una forma sencilla e intuitiva de interactuar con el robot. El sistema permite controlar las diferentes funcionalidades disponibles, visualizar información en tiempo real y supervisar el estado general del robot desde cualquier navegador conectado a la misma red.

El diseño se ha desarrollado utilizando HTML, CSS y JavaScript, siguiendo una estructura modular que facilita el mantenimiento y la ampliación de nuevas funcionalidades. Además, la interfaz se adapta a distintos perfiles de usuario, mostrando únicamente las opciones correspondientes a cada rol.



5.2.1 Landing page

La landing page constituye el punto de entrada principal al sistema Patricio. Su objetivo es presentar el proyecto, mostrar sus principales funcionalidades y proporcionar acceso a las opciones de registro e inicio de sesión. El diseño está orientado tanto a padres como a educadores interesados en conocer las capacidades del robot antes de acceder a la plataforma.

La figura 2 muestra la cabecera y la sección principal de presentación del proyecto, donde se destaca el propósito educativo del robot y su orientación al aprendizaje infantil.

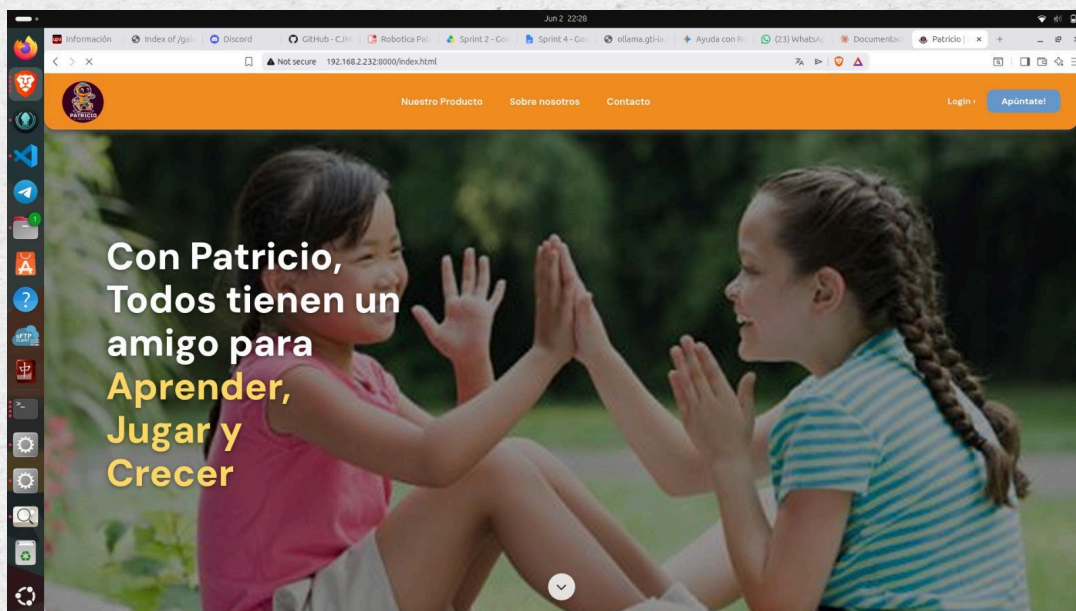


Figura 2. Página principal (landing page) de la plataforma web Patricio (1)

En la figura 3 se observa la sección dedicada a describir las características principales del robot, incluyendo navegación autónoma, actividades educativas e interacción inteligente.



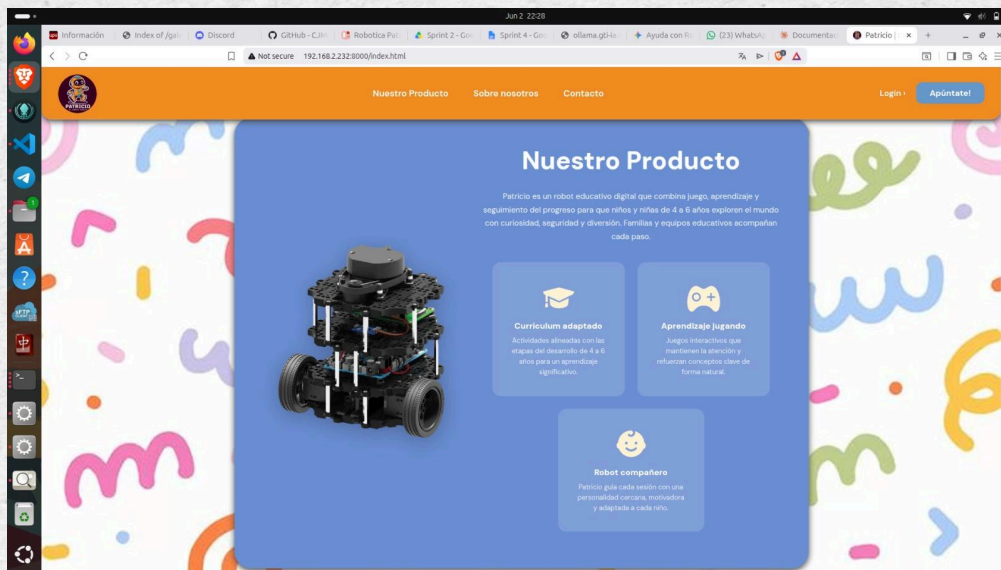


Figura 3. Página principal (landing page) de la plataforma web Patricio (2)

La figura 4 muestra la información del equipo desarrollador y los integrantes responsables del proyecto.

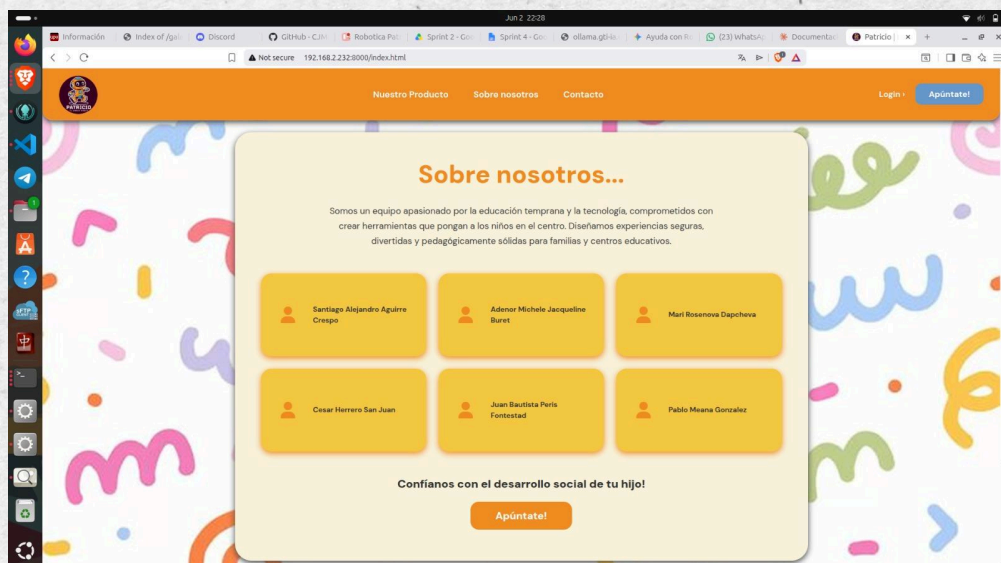


Figura 4. Página principal (landing page) de la plataforma web Patricio (3)

La figura 5 presenta los distintos medios de contacto disponibles para los usuarios interesados en obtener más información sobre el proyecto.



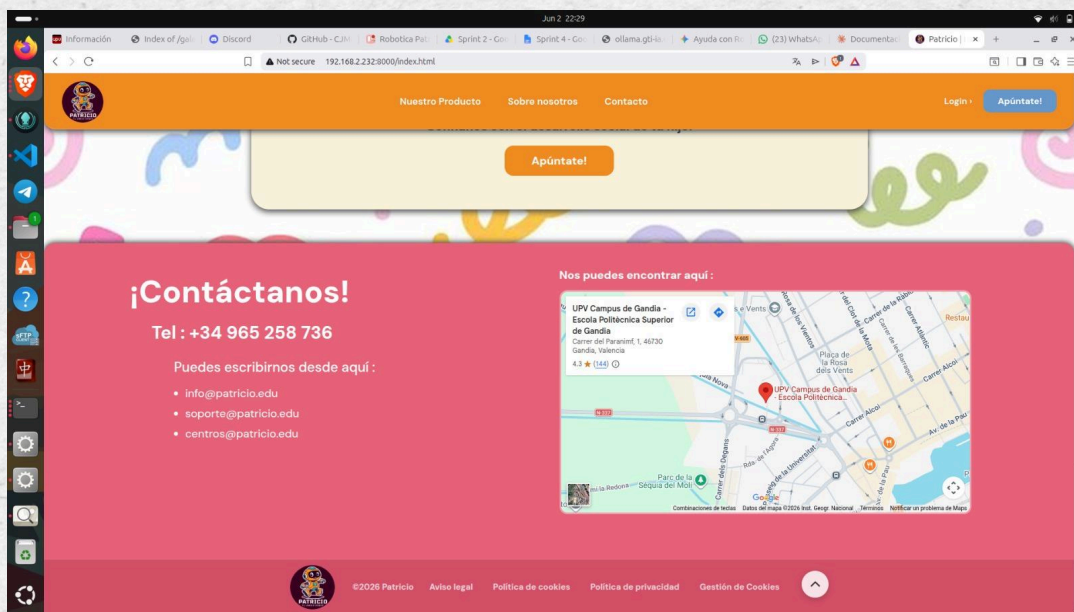


Figura 5. Página principal (landing page) de la plataforma web Patricio (4)

5.2.2 Pantalla de registro

La pantalla de registro permite crear nuevas cuentas dentro de la plataforma. Durante este proceso se recopilan los datos necesarios para identificar al usuario y permitir posteriormente el acceso a las funcionalidades del sistema.

La figura 6 muestra el formulario utilizado para introducir los datos de registro y crear una nueva cuenta en el sistema.

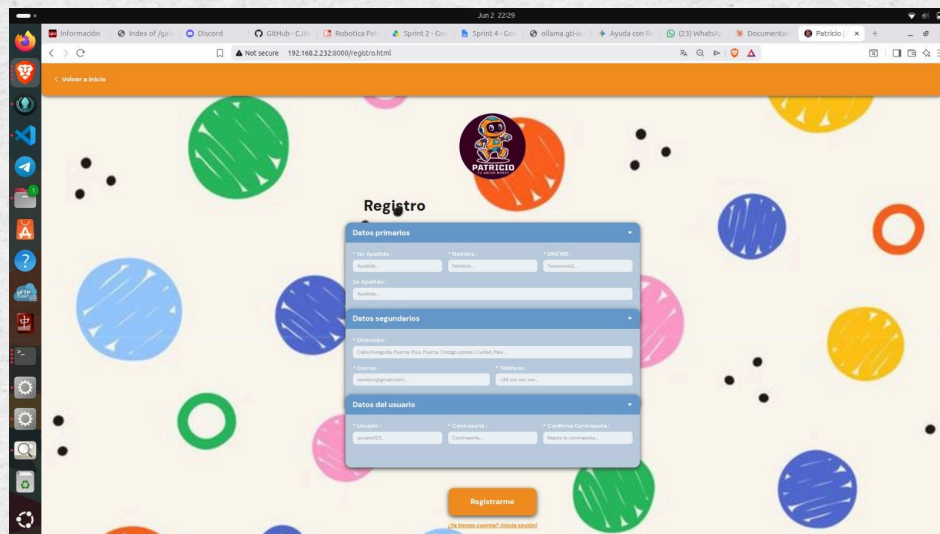


Figura 6. Pantalla de registro de nuevo usuario en la interfaz web de Patricio.

5.2.3 Pantalla de Inicio de sesión

La autenticación de usuarios se realiza mediante una pantalla de inicio de sesión que valida las credenciales almacenadas en la base de datos. Como puede observarse en la figura 7, el usuario debe introducir su nombre de usuario y contraseña para acceder a la plataforma.

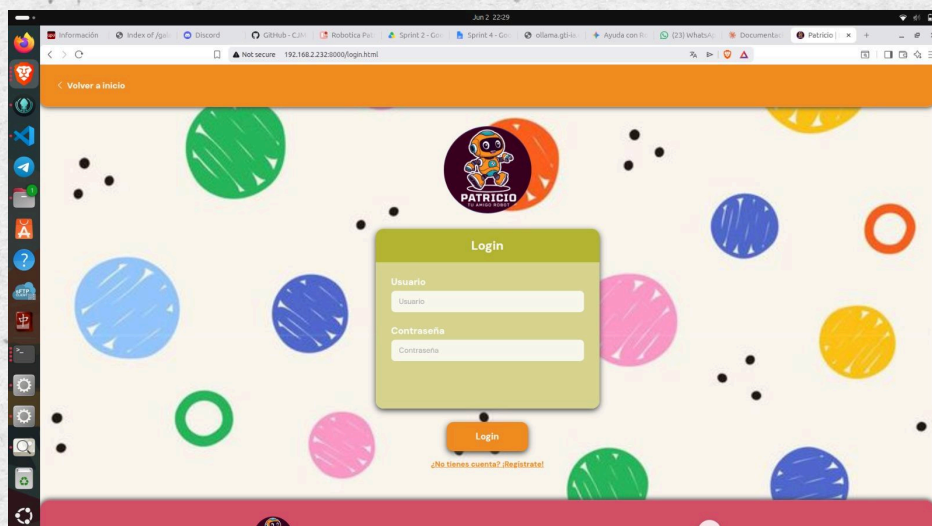


Figura 7. Pantalla de login en la interfaz web de Patricio.

5.2.4 Pantalla de Panel de control

El panel de control centraliza las funcionalidades principales de supervisión y control del robot. Desde esta pantalla es posible visualizar información en tiempo real y acceder a las diferentes actividades disponibles.

La figura 8 muestra la interfaz principal utilizada por los usuarios para supervisar el estado general del sistema, visualizar la cámara y acceder a las funcionalidades de control.

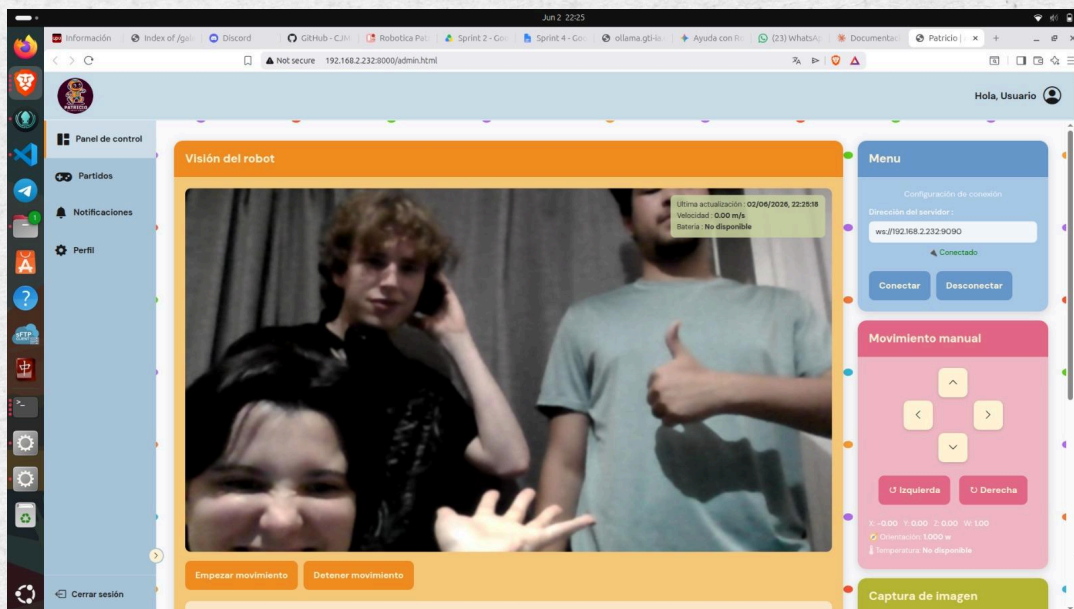


Figura 8. Vista general del panel de control para tutor/empleador.

En la figura 9 se observa la sección destinada a iniciar y supervisar las actividades educativas implementadas en el robot.



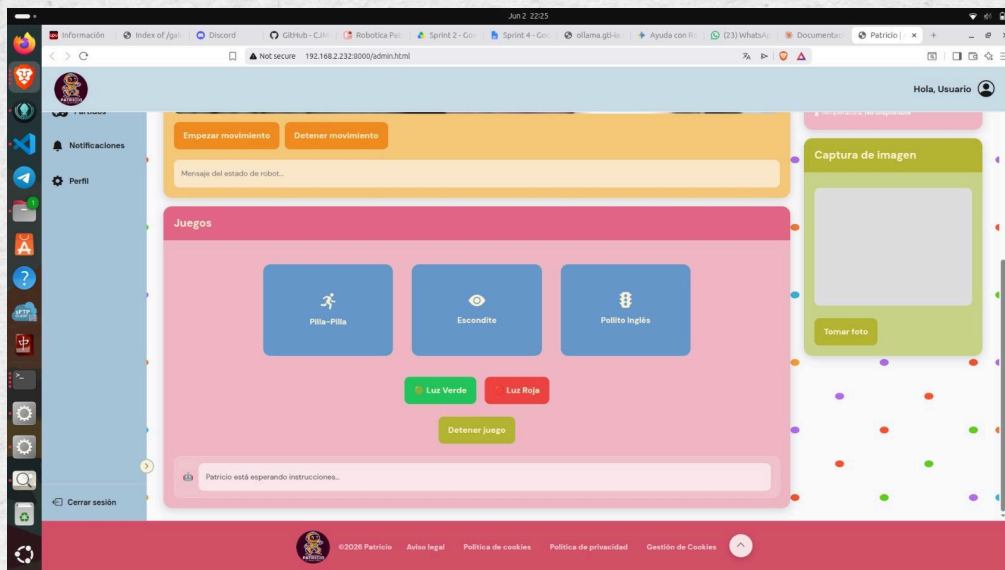


Figura 9.Sección de gestión y listado de partidas en el panel de control.

5.2.5 Pantalla de Sección de Partidos

Esta sección permite consultar el historial de partidas realizadas, así como la información asociada a cada actividad ejecutada por el robot.

La figura 10 muestra la interfaz utilizada para consultar y administrar las partidas registradas en el sistema.

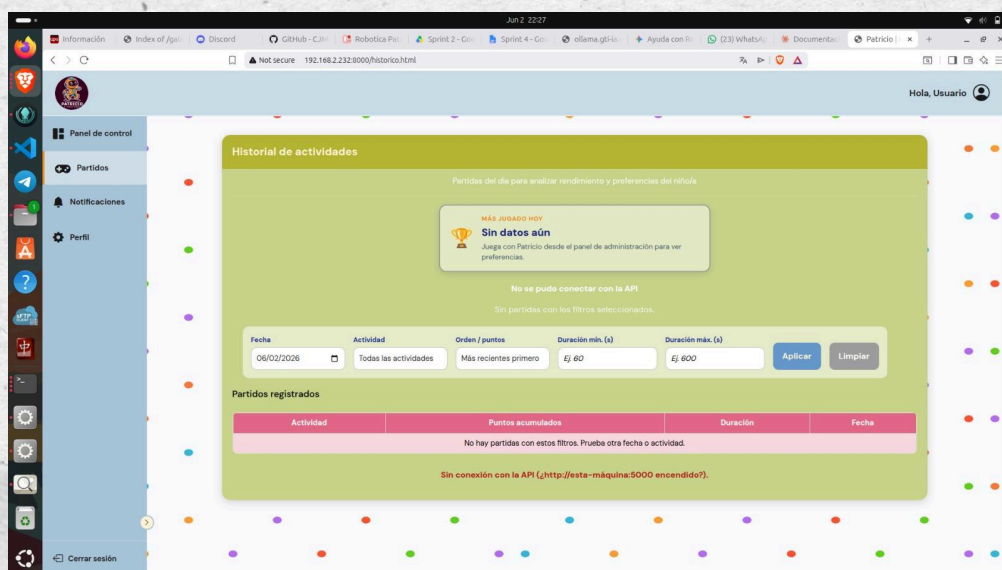


Figura 10. Sección de notificaciones e incidencias en la interfaz web.

5.2.6 Pantalla de Sección de Notificaciones

La plataforma incorpora un sistema de notificaciones destinado a informar sobre eventos relevantes ocurridos durante la ejecución del sistema.

La figura 11 presenta la interfaz utilizada para visualizar incidencias, avisos y eventos generados por el robot.

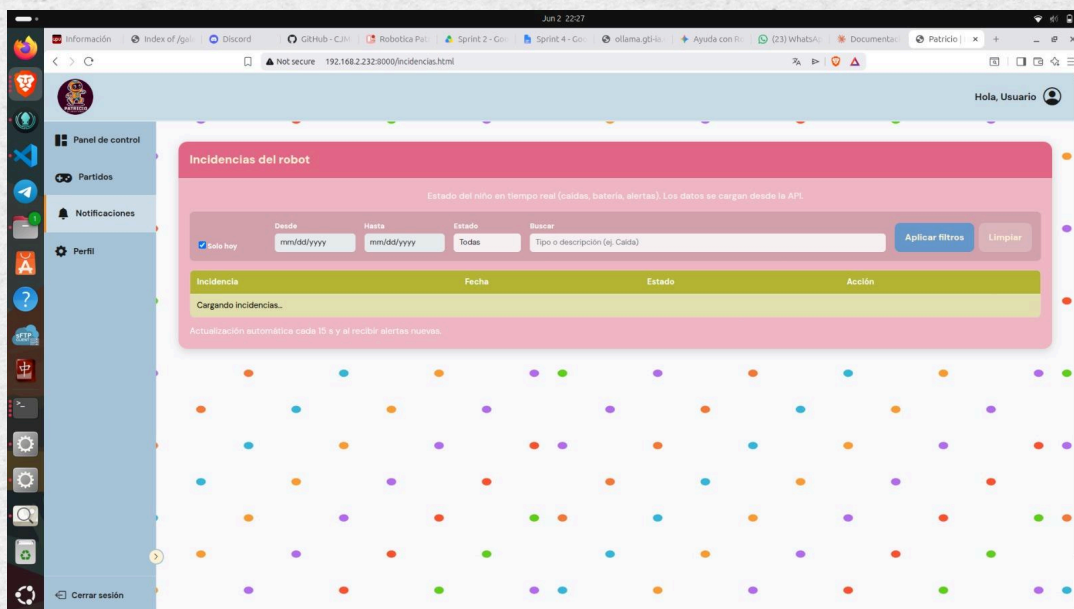


Figura 11. Vista del perfil de usuario dentro de la plataforma Patricio.

5.2.7 Pantalla de la Sección de Perfil

La gestión del perfil permite modificar los datos personales del usuario y actualizar la información asociada a su cuenta.

Como se muestra en la figura 12, el usuario puede actualizar sus datos personales mediante un formulario accesible desde la propia plataforma.



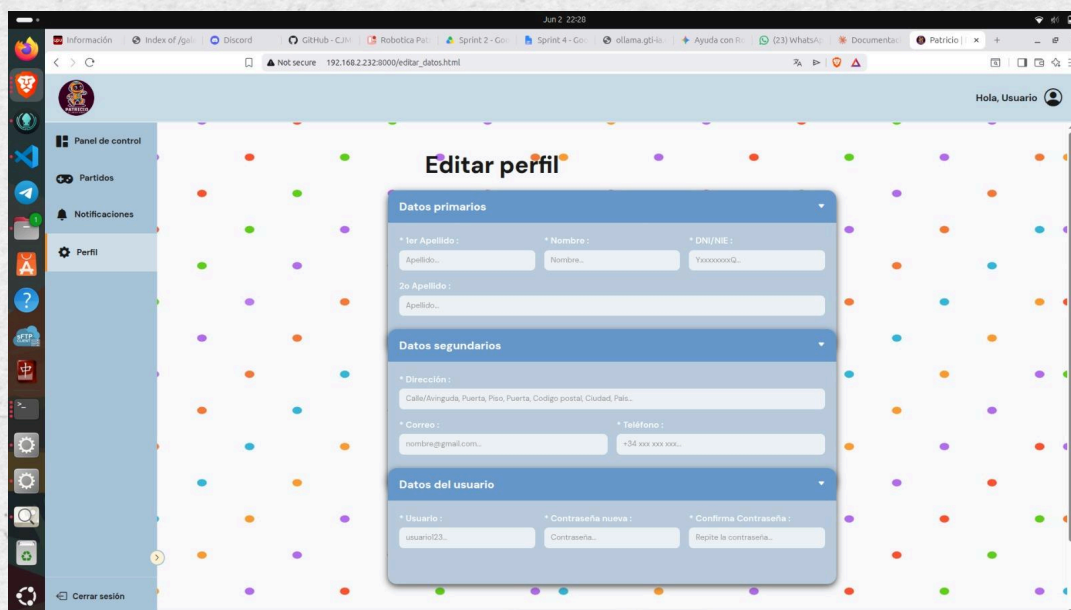


Figura 12. Edición de datos personales y configuración del perfil de usuario.

5.3 Diseño del back-end

El back-end de Patricio es el encargado de gestionar la lógica del sistema, la persistencia de datos y la comunicación entre la interfaz web y los distintos módulos ROS 2. Su diseño se basa en una arquitectura modular compuesta por una API desarrollada en Flask, una base de datos MySQL y diversos servicios responsables de la navegación, visión artificial, reconocimiento e interacción con el usuario.

La API actúa como punto central de comunicación, procesando las peticiones realizadas desde la interfaz web y coordinando las operaciones necesarias sobre la base de datos y los nodos ROS 2. Gracias a esta arquitectura, es posible desacoplar la lógica de negocio de la interfaz gráfica y facilitar futuras ampliaciones del sistema.

5.3.1 Base de datos

Con el objetivo de almacenar de forma estructurada la información generada durante el funcionamiento del sistema, se ha diseñado una base de datos relacional utilizando MySQL. La estructura permite gestionar los usuarios registrados, las actividades disponibles, las partidas realizadas y las incidencias detectadas durante el uso del robot.

Como se puede observar en la Figura 13, el modelo de datos está formado por cuatro entidades principales: **Usuarios**, **Actividades**, **Partidas** e **Incidencias**, relacionadas mediante claves foráneas que garantizan la integridad de la información almacenada.

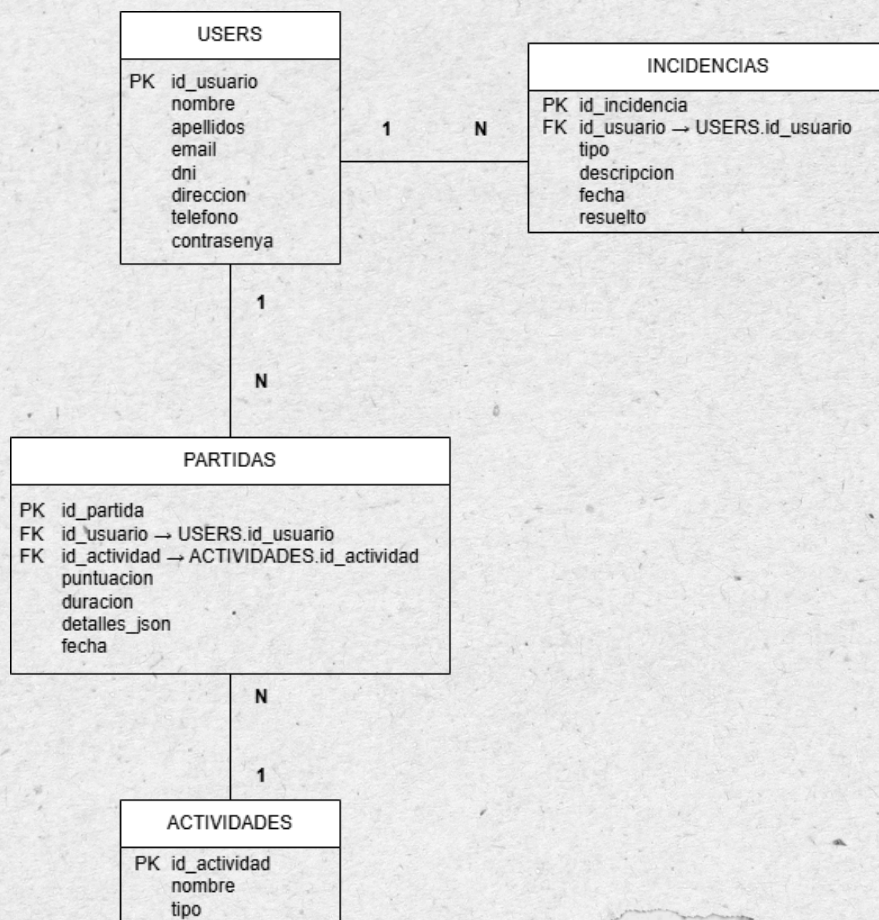


Figura 13. Diagrama Entidad-Relación de la base de datos Patricio



Usuarios almacena la información de los padres o tutores que utilizan el sistema, incluyendo datos necesarios para la autenticación y gestión de la cuenta. Cada usuario puede realizar múltiples actividades y tener asociadas diferentes incidencias a lo largo del tiempo.

La entidad **Actividades** contiene el catálogo de funcionalidades disponibles en el robot, incluyendo juegos y actividades educativas. Cada actividad dispone de un identificador único y una clasificación que permite organizar las distintas funcionalidades implementadas.

La entidad **Partidas** registra cada ejecución realizada por un usuario. Además de relacionar al usuario con la actividad ejecutada, almacena información adicional como la fecha, duración, puntuación obtenida o datos específicos de la actividad mediante campos estructurados.

Por último, la entidad **Incidencias** permite registrar eventos relevantes detectados durante el funcionamiento del sistema, como errores o situaciones anómalas, facilitando su seguimiento y posterior análisis.

Gracias a esta estructura, el sistema puede mantener un historial completo de uso del robot, almacenar resultados de las actividades realizadas y gestionar la información necesaria para futuras ampliaciones del proyecto.

5.3.2 API Flask

La comunicación entre la interfaz web, la base de datos y el sistema robótico se centraliza mediante una API desarrollada con Flask. Esta API actúa como intermediaria entre los distintos componentes del proyecto, permitiendo separar la lógica de negocio de la interfaz de usuario y facilitando el mantenimiento del sistema.

Como se observa en la Figura 14, todas las peticiones realizadas desde la interfaz web son enviadas a la API mediante solicitudes HTTP utilizando formato JSON. La API procesa estas solicitudes y realiza las operaciones necesarias sobre la base de datos o sobre los servicios del robot.



Entre las principales funcionalidades implementadas se encuentran la gestión de usuarios, la autenticación mediante inicio de sesión, el registro de partidas realizadas por los usuarios y el almacenamiento de incidencias detectadas durante el funcionamiento del robot.

Además, la API también participa en la comunicación con ROS 2. A través de los distintos endpoints implementados, la interfaz web puede iniciar o detener actividades, consultar el estado del sistema y enviar órdenes al robot sin necesidad de interactuar directamente con los nodos ROS.

Esta arquitectura permite desacoplar completamente la capa de presentación de la lógica interna del sistema, mejorando la escalabilidad, la reutilización de componentes y la facilidad de mantenimiento del proyecto.

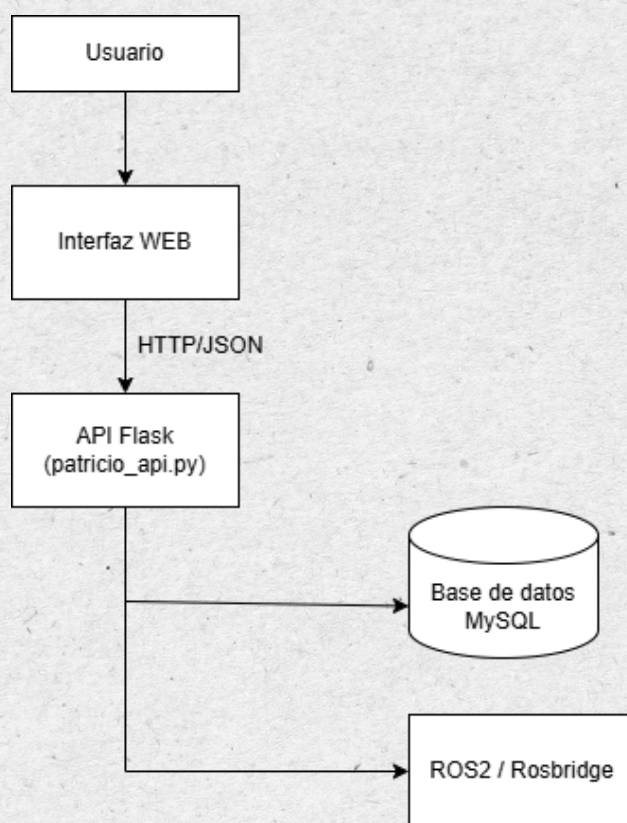


Figura 14. Flujo de comunicación entre la interfaz web, la API Flask, la base de datos MySQL y ROS 2.

5.3.3 Sistema de visión artificial

Con el objetivo de dotar al robot de capacidades de percepción del entorno y de interacción con el usuario, se ha desarrollado un sistema de visión artificial basado en OpenCV y MediaPipe. Este sistema permite capturar imágenes procedentes de la cámara del robot, procesarlas y generar información que posteriormente es utilizada por los distintos juegos y actividades implementados.

La arquitectura general del sistema de visión se muestra en la Figura 15. Como puede observarse, el procesamiento se organiza en varias capas independientes que permiten separar la adquisición de imágenes, la detección de personas y el consumo de la información por parte de los distintos módulos del sistema.

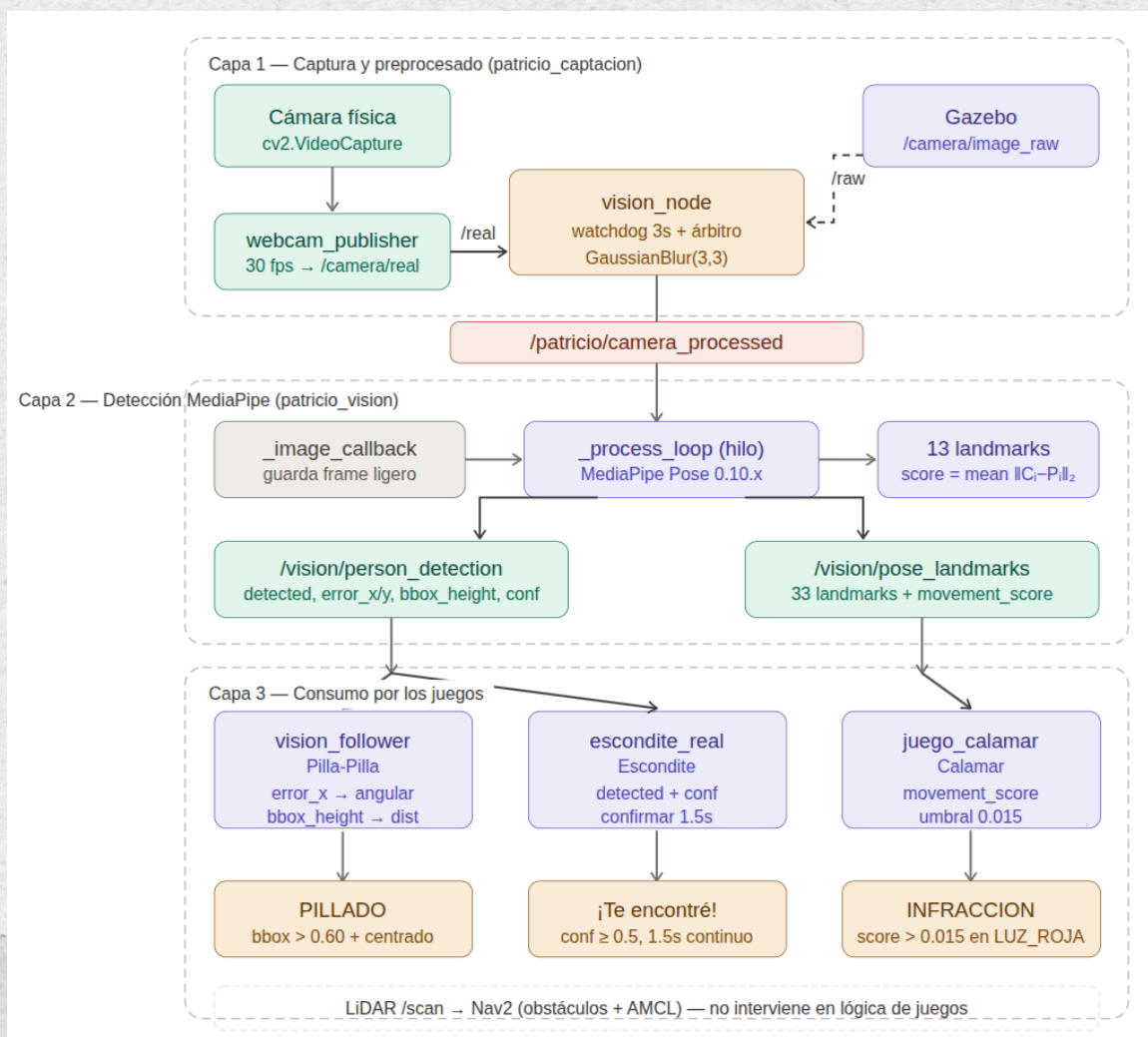


Figura 15. Arquitectura completa de la lógica de visión artificial.

Captura y preprocesado de imagen

La primera etapa del sistema corresponde a la adquisición y preparación de las imágenes procedentes de la cámara. Tal y como se muestra en la Figura 16, el módulo de captación es el encargado de recibir los fotogramas, realizar operaciones básicas de preprocesado y publicar una imagen unificada para el resto de componentes.

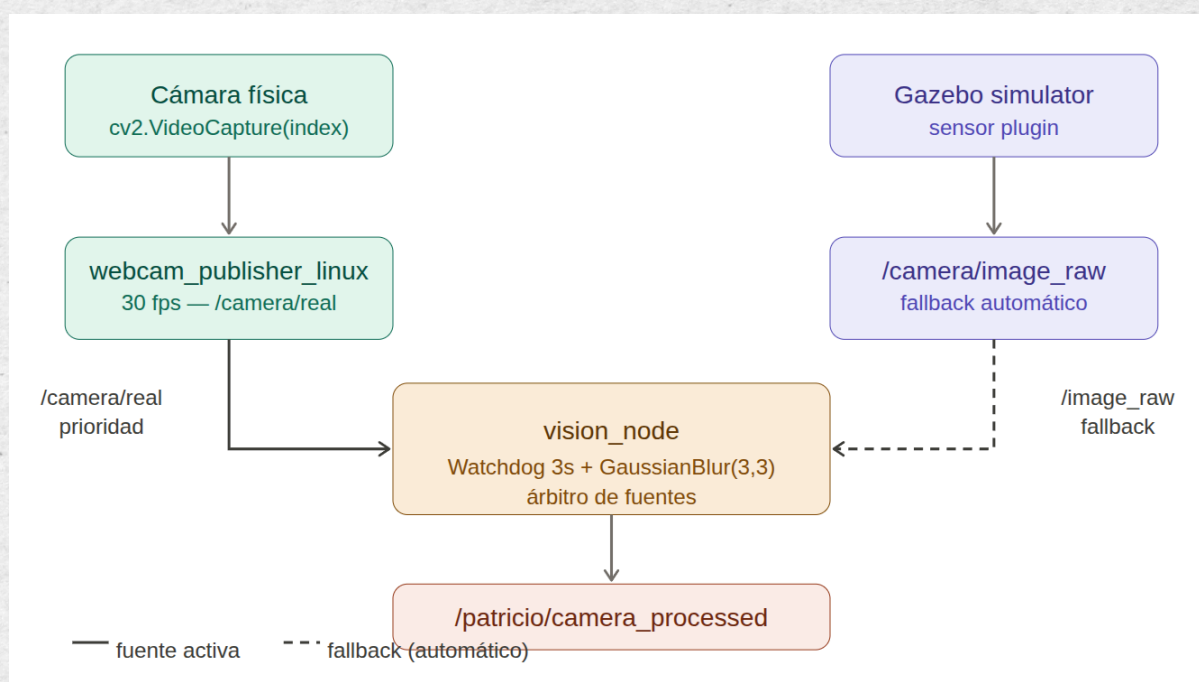


Figura 16. Capa 1 de la lógica de visión artificial.

Esta capa permite abstraer el origen de la imagen, pudiendo trabajar tanto con la cámara física del robot como con la cámara del simulador Gazebo sin modificar el resto de la arquitectura.

Detección mediante MediaPipe

La segunda capa incorpora MediaPipe como sistema de detección corporal. A partir de las imágenes procesadas, el sistema identifica la presencia de personas

y obtiene información sobre la posición y movimiento de diferentes partes del cuerpo.

Como se observa en la Figura 17, esta etapa genera información estructurada que posteriormente puede ser utilizada por otros módulos del robot.

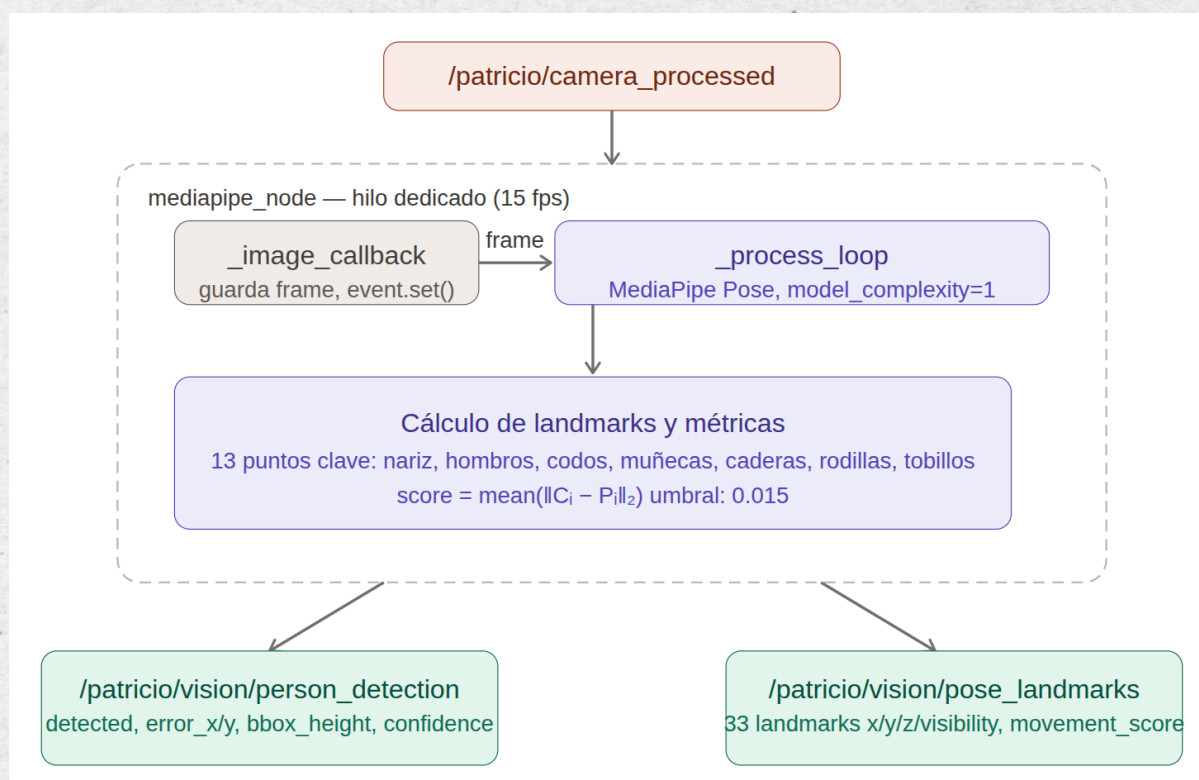


Figura 17. Capa 2 de la lógica de visión artificial.

Entre los datos obtenidos destacan la detección de personas, la estimación de la posición corporal y el análisis de movimiento, elementos fundamentales para el funcionamiento de varios juegos y actividades.

Integración con los juegos

La información generada por las capas anteriores es consumida por los distintos módulos del sistema. Tal y como se representa en la Figura 18, los juegos implementados utilizan estos datos para detectar la presencia del usuario, validar movimientos y adaptar su comportamiento en función de la interacción observada.

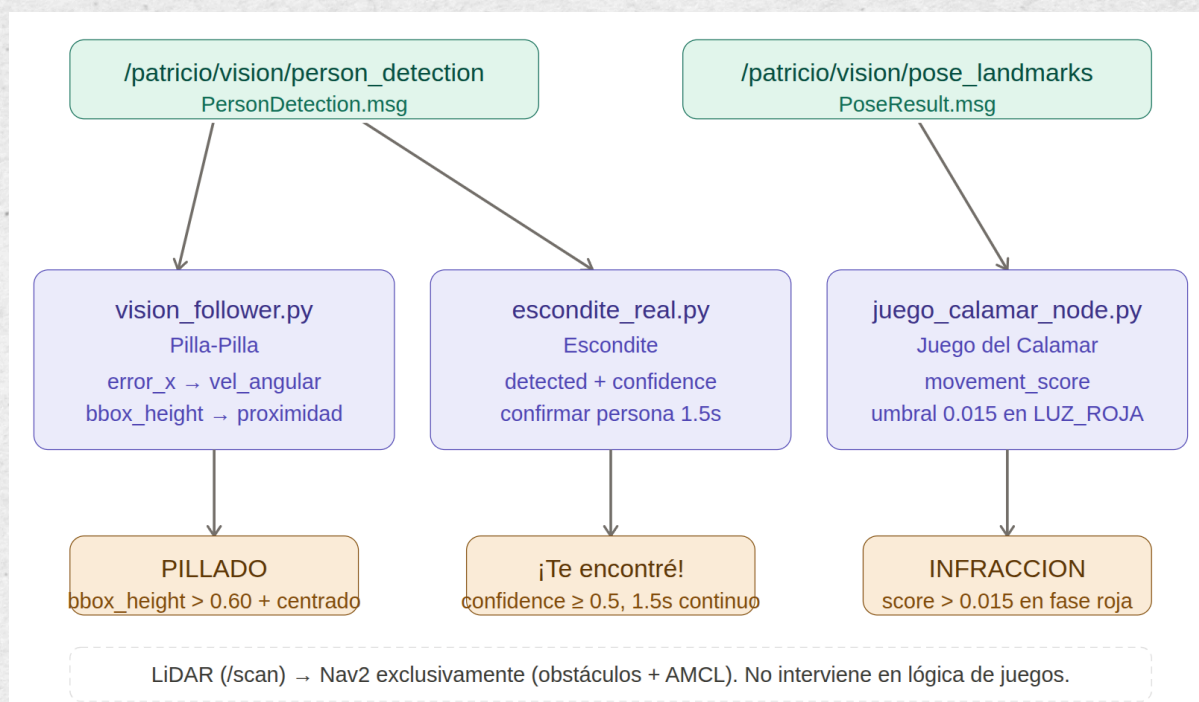


Figura 18. Capa 3 de la lógica de visión artificial.

Gracias a esta arquitectura modular, el sistema de visión puede reutilizarse en diferentes actividades sin necesidad de duplicar procesamiento, facilitando además futuras ampliaciones y mejoras.

5.3.4 Inteligencia artificial y reconocimiento

Con el objetivo de ofrecer una interacción más natural entre el usuario y el robot, se ha incorporado un sistema de inteligencia artificial conversacional capaz de procesar comandos de voz y generar respuestas adaptadas al contexto de uso. Este sistema permite que Patricio pueda comunicarse con los niños mediante lenguaje natural, mejorando la experiencia de interacción durante las distintas actividades educativas.

La arquitectura general del sistema se muestra en la Figura 19. El proceso comienza con la captura de audio mediante un micrófono, que posteriormente es procesado por un sistema de reconocimiento de voz para obtener el texto correspondiente. Una vez generado el texto, este se envía al modelo de

inteligencia artificial encargado de interpretar la intención del usuario y generar una respuesta adecuada.

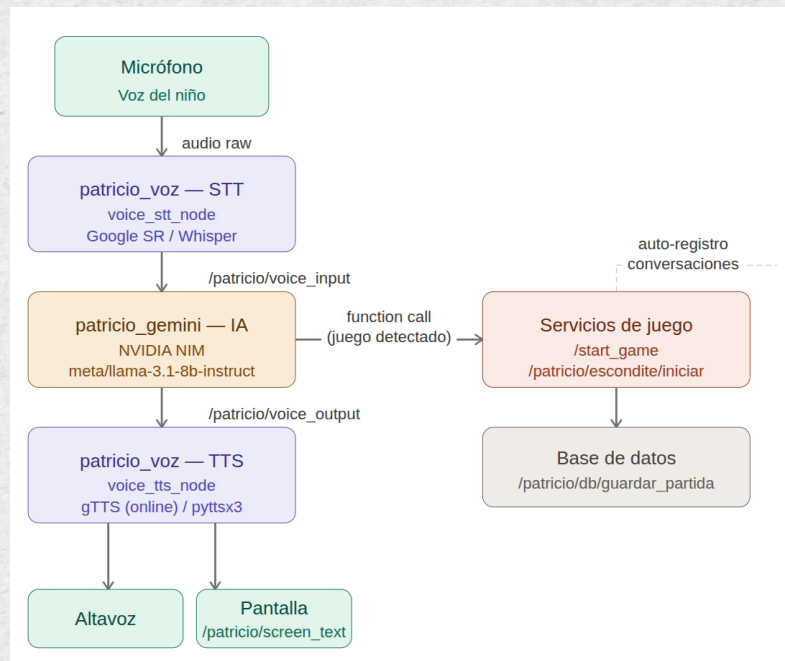


Figura 19. Pipeline completo del sistema de voz e inteligencia artificial.

Además de generar respuestas conversacionales, el sistema puede identificar determinadas órdenes relacionadas con el funcionamiento del robot, como el inicio de actividades o la ejecución de acciones específicas. Esto permite que algunas funcionalidades puedan activarse directamente mediante comandos de voz, facilitando la interacción para los usuarios más pequeños.

Una vez generada la respuesta, esta es transformada nuevamente en audio mediante un sistema de síntesis de voz, permitiendo que el robot responda verbalmente al usuario. De esta forma se establece un ciclo completo de comunicación basado en voz, desde la captura inicial del mensaje hasta la reproducción de la respuesta generada.

La integración de este sistema con ROS 2 permite que las órdenes interpretadas por la inteligencia artificial puedan traducirse en acciones concretas dentro del robot, manteniendo una arquitectura modular y desacoplada del resto de componentes del sistema.

Gracias a esta funcionalidad, Patricio es capaz de ofrecer una interacción más cercana y dinámica, complementando las capacidades de navegación, visión artificial y juegos educativos desarrolladas durante el proyecto.

5.3.5 Sistema de emociones

Con el objetivo de mejorar la interacción entre el robot y los niños, se ha desarrollado un sistema de expresiones faciales capaz de representar diferentes estados emocionales mediante una pantalla integrada en el robot. Este sistema permite complementar la comunicación verbal con elementos visuales fácilmente reconocibles por los usuarios.

Las expresiones faciales forman parte de la identidad visual de Patricio y tienen como finalidad transmitir información sobre el estado actual del robot de una manera intuitiva. Gracias a ello, los niños pueden interpretar con mayor facilidad las acciones y respuestas generadas por el sistema.

Las emociones implementadas se representan mediante diferentes expresiones gráficas mostradas en la pantalla frontal del robot. Entre ellas se encuentran estados de felicidad, tristeza, alerta y concentración, asociados a distintas situaciones durante la ejecución de actividades y juegos.

La Figura X muestra la representación de la emoción de felicidad utilizada cuando el robot completa correctamente una actividad o mantiene una interacción positiva con el usuario.





Figura 20. Representación de la emoción Feliz en la interfaz del robot Patricio.

La Figura 21 presenta la emoción de tristeza, utilizada en situaciones donde el robot detecta errores o no consigue completar una determinada acción.



Figura 21. Representación de la emoción Triste en la interfaz del robot Patricio.

La Figura 22 corresponde al estado de alerta, empleado cuando el sistema detecta una situación que requiere atención o supervisión.

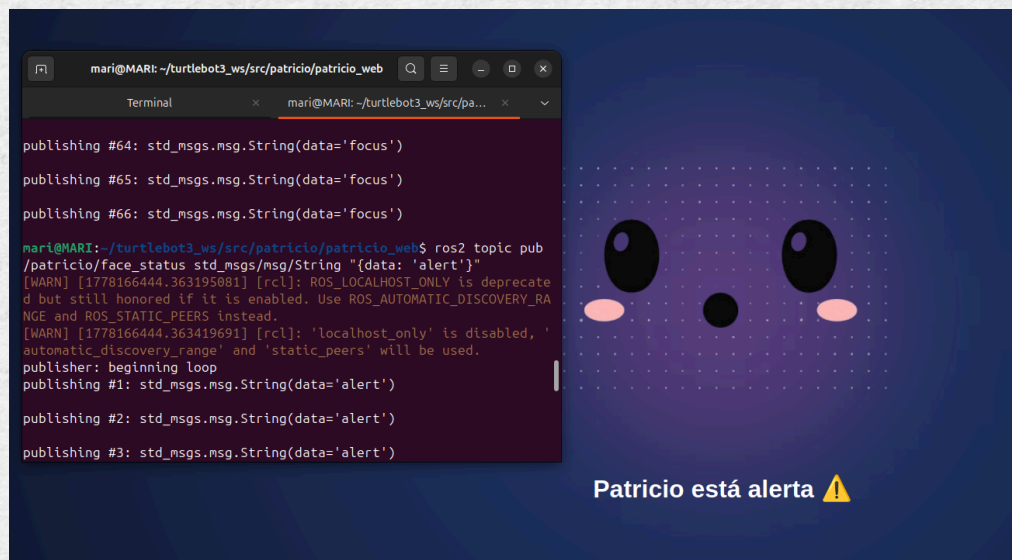


Figura 22. Representación de la emoción En alerta en la interfaz del robot Patricio.

Por último, la Figura 23 muestra la emoción de concentración, utilizada durante la ejecución de tareas que requieren procesamiento o seguimiento de objetivos.



Figura 23. Representación de la emoción Concentrado en la interfaz del robot Patricio.



La incorporación de este sistema de emociones permite humanizar la interacción con el robot y mejorar la experiencia de uso, especialmente en el contexto educativo para el que ha sido diseñado el proyecto.

5.4 Explicación de las tecnologías necesarias

El desarrollo del proyecto Patricio ha requerido la integración de diversas tecnologías de software orientadas a la robótica, la visión artificial, la inteligencia artificial y el desarrollo web. La Tabla 3 resume las principales herramientas utilizadas y su función dentro del sistema.

Tecnología	Función dentro del proyecto
Ubuntu 24.04	Sistema operativo utilizado para el desarrollo y ejecución del robot.
ROS 2 Jazzy	Middleware robótico encargado de la comunicación entre nodos y la gestión del sistema.
Gazebo	Simulador utilizado para realizar pruebas en entornos virtuales antes de la ejecución en el robot real.
Nav2	Framework de navegación autónoma empleado para la localización, planificación de rutas y evitación de obstáculos.
Python	Lenguaje principal utilizado para el desarrollo de nodos ROS 2 y servicios del sistema.
Flask	Framework utilizado para implementar la API web y la comunicación entre la interfaz y el robot.
MySQL	Sistema gestor de bases de datos empleado para almacenar usuarios, partidas, actividades e incidencias.



HTML, CSS y JavaScript	Tecnologías utilizadas para el desarrollo de la interfaz web.
Rosbridge	Servidor encargado de conectar ROS 2 con aplicaciones web mediante WebSockets.
Roslibjs	Librería JavaScript utilizada para interactuar con ROS 2 desde el navegador.
OpenCV	Biblioteca utilizada para el procesamiento de imágenes y visión artificial.
MediaPipe	Framework empleado para la detección corporal y el análisis de movimiento.
NVIDIA NIM	Plataforma utilizada para la integración de inteligencia artificial conversacional.
NVIDIA NIM	Herramientas utilizadas para la síntesis de voz del robot.

Tabla 3. Tecnologías utilizadas en el desarrollo del proyecto Patricio y su función dentro del sistema.

La combinación de estas tecnologías ha permitido desarrollar una solución capaz de integrar navegación autónoma, visión artificial, interacción mediante voz y control remoto desde una interfaz web, cumpliendo los objetivos establecidos para el proyecto.

6. Implementación

6.1 Estructura de carpetas del proyecto

El desarrollo de Patricio se ha organizado siguiendo una arquitectura modular basada en paquetes ROS 2. Esta estructura permite separar las distintas responsabilidades del sistema, facilitando el mantenimiento, la reutilización de componentes y el desarrollo independiente de nuevas funcionalidades.

La estructura principal del proyecto se compone de los siguientes paquetes:



- **patricio_my_world**, encargado de la simulación del entorno y de los modelos utilizados en Gazebo.
- **patricio_nav_punto**, responsable de la navegación autónoma mediante Nav2.
- **patricio_captacion**, encargado de la adquisición y preprocesado de imágenes procedentes de las cámaras.
- **patricio_interfaces**, que contiene los mensajes y servicios personalizados utilizados por el sistema.
- **patricio_pilla_pilla**, que implementa la lógica del juego Pilla-Pilla.
- **patricio_escondite**, encargado de la implementación del juego Escondite.
- **patricio_calamar**, que implementa la lógica del Juego del Calamar.
- **patricio_web**, responsable de la interfaz web y de la comunicación con los usuarios.

Esta organización permite mantener una separación clara entre la simulación, la navegación, la visión artificial, los juegos educativos y la interfaz de usuario, reduciendo las dependencias entre componentes y facilitando futuras ampliaciones del proyecto.

6.2 Implementación de los paquetes

6.2.1 Simulación del entorno

La simulación del entorno se implementó mediante el paquete **patricio_my_world**, desarrollado sobre Gazebo Harmonic. Este paquete contiene los modelos tridimensionales necesarios para representar el entorno de pruebas, incluyendo el modelo del TurtleBot3 y los distintos elementos que componen el escenario virtual.

El entorno se inicializa mediante archivos *launch* encargados de arrancar Gazebo, generar el robot dentro de la simulación y publicar el estado de sus articulaciones a través de ROS 2. Gracias a ello, es posible realizar pruebas de navegación, visión artificial e interacción con los distintos juegos sin necesidad de utilizar continuamente el robot físico.



Además del entorno virtual, este paquete incorpora los modelos y configuraciones necesarias para la comunicación entre Gazebo y ROS 2, permitiendo que los sensores simulados generen información equivalente a la obtenida en el robot real. Entre los datos utilizados por el resto de componentes destacan las imágenes de cámara, la información de odometría y los datos del sensor LiDAR empleados durante la navegación autónoma.

La utilización de un entorno simulado ha permitido acelerar el desarrollo del proyecto, facilitando la validación de funcionalidades y la detección de errores antes de su despliegue sobre el robot físico.

6.2.2 Navegación autónoma

La navegación autónoma se implementó mediante el paquete `patricio_nav_punto`, utilizando Nav2 para la localización, planificación de rutas y desplazamiento del robot dentro del entorno.

Este módulo emplea componentes como AMCL para la localización probabilística y los servidores de planificación y control de Nav2 para calcular y ejecutar rutas de forma autónoma. La comunicación se realiza principalmente mediante los tópicos `/cmd_vel`, `/odom` y `/map`, así como mediante la acción `/navigate_to_pose`, utilizada para enviar objetivos de navegación.

Esta funcionalidad es utilizada por diferentes actividades del proyecto, permitiendo que el robot se desplace de forma autónoma durante los juegos y tareas educativas.

6.2.3 Captación y procesamiento de imagen

La adquisición de imágenes se implementó mediante el paquete `patricio_captacion`, encargado de gestionar las distintas fuentes de vídeo disponibles en el sistema.

Las imágenes capturadas son procesadas utilizando OpenCV para aplicar operaciones básicas de preprocesado y posteriormente se publican en el tópico `/patricio/camera_processed`, que actúa como punto de entrada para el resto de módulos de visión artificial.



Esta arquitectura permite trabajar indistintamente con la cámara física del robot o con la cámara proporcionada por el simulador.

6.2.4 Interfaz web

La interacción con el robot se realiza mediante el paquete `patricio_web`, encargado de proporcionar la interfaz gráfica accesible desde el navegador.

La aplicación permite iniciar actividades, consultar información del sistema y supervisar el estado del robot en tiempo real. La comunicación con ROS 2 se realiza mediante Rosbridge y la librería `roslibjs`, que permiten intercambiar información utilizando WebSockets sin necesidad de acceder directamente al entorno ROS.

Gracias a esta arquitectura, los usuarios pueden controlar el robot de forma sencilla desde cualquier dispositivo conectado a la red.

6.2.5 Juegos educativos

Los juegos educativos se implementaron mediante paquetes independientes para cada actividad: `patricio_pilla_pilla`, `patricio_escondite` y `patricio_calamar`. Esta organización permite mantener separada la lógica de cada juego y facilita la incorporación de nuevas actividades en el futuro.

La comunicación con el resto del sistema se realiza mediante servicios ROS 2 definidos en el paquete `patricio_interfaces`, destacando `StartGame.srv`, utilizado para iniciar las distintas actividades desde la interfaz web.

Además de la navegación autónoma, estos juegos utilizan información procedente de los módulos de visión artificial para detectar la presencia y los movimientos de los usuarios durante la partida.



6.2.6 Persistencia de datos

La persistencia de información se implementó mediante el paquete `patricio_db_interface`, encargado de conectar los nodos ROS 2 con la base de datos MySQL.

Este módulo expone servicios que permiten registrar partidas e incidencias generadas durante el funcionamiento del sistema. Internamente, las peticiones son enviadas a una API desarrollada en Flask, responsable de procesar la información y almacenarla en la base de datos.

Gracias a esta arquitectura, los distintos componentes del robot pueden guardar información sin necesidad de conocer los detalles de la comunicación HTTP o de la estructura interna de la base de datos.

6.3 Uso de OpenCV

OpenCV ha sido una de las principales tecnologías utilizadas para implementar las funcionalidades de visión artificial del proyecto. Su integración ha permitido capturar imágenes procedentes de las cámaras del sistema, realizar tareas de preprocesado y proporcionar una entrada común para los distintos módulos encargados de analizar el entorno.

La combinación de OpenCV con ROS 2 ha facilitado el intercambio de imágenes entre nodos, permitiendo que los distintos componentes del sistema puedan acceder a la información visual de forma eficiente. Sobre esta base se han construido las funcionalidades de detección y seguimiento utilizadas por los juegos educativos y por los sistemas de interacción implementados en el robot.

6.3.1 Captación y preprocesado de imágenes

La primera funcionalidad implementada mediante OpenCV corresponde a la adquisición y preparación de imágenes. Para ello se emplea la clase `cv2.VideoCapture`, encargada de obtener los fotogramas procedentes de la cámara física del robot o de la cámara disponible en el simulador.



Como se puede observar en la Figura 24, las imágenes capturadas pueden visualizarse en tiempo real desde la interfaz web del sistema, permitiendo supervisar el entorno y comprobar el correcto funcionamiento de los módulos de visión artificial.

Una vez capturadas, las imágenes son sometidas a diferentes operaciones de preprocesado con el objetivo de mejorar la estabilidad del procesamiento posterior. Entre estas operaciones destaca la aplicación de filtros para reducir el ruido presente en los fotogramas y facilitar el trabajo de los módulos de detección.

Tras este proceso, las imágenes son publicadas en ROS 2 para que puedan ser consumidas por el resto de componentes del sistema. De esta forma se consigue una arquitectura modular en la que la adquisición de imágenes queda desacoplada de los algoritmos encargados de analizar la información visual.



Figura 24. Visualización en tiempo real de la cámara del robot desde la interfaz web.

6.3.2 Detección corporal mediante MediaPipe

Una vez obtenidas y procesadas las imágenes mediante OpenCV, estas son utilizadas por el sistema de detección corporal basado en MediaPipe Pose. Esta herramienta permite identificar la posición del cuerpo humano a partir de una serie de puntos de referencia (*landmarks*) distribuidos sobre diferentes articulaciones y zonas del cuerpo.

El sistema analiza continuamente los fotogramas recibidos y calcula la posición de los principales puntos corporales, generando información sobre la postura, el movimiento y la localización de la persona dentro de la imagen. Estos datos son posteriormente publicados en ROS 2 para que puedan ser utilizados por los distintos módulos del proyecto.

La detección corporal desarrollada permite determinar si una persona está presente en la escena, estimar su posición relativa respecto al robot y medir el nivel de movimiento realizado. Esta información resulta fundamental para el funcionamiento de actividades como Pilla-Pilla, Escondite y Juego del Calamar, ya que permite adaptar el comportamiento del robot en función de las acciones realizadas por el usuario.

Gracias a la integración entre OpenCV y MediaPipe, el sistema es capaz de realizar estas tareas en tiempo real, proporcionando una respuesta rápida y adecuada durante la interacción con el niño.

6.3.3 Integración con los juegos

La información obtenida mediante OpenCV y MediaPipe se integra directamente en varios de los juegos implementados en Patricio. Gracias al análisis de la imagen en tiempo real, el robot puede detectar la presencia del usuario y adaptar su comportamiento en función de sus movimientos.

En el juego Pilla-Pilla, los datos de detección permiten estimar la posición relativa del niño respecto al robot, facilitando el seguimiento durante la partida. En Escondite, el sistema utiliza la detección corporal para comprobar si el



usuario ha sido encontrado, mientras que en el Juego del Calamar se analiza el nivel de movimiento para detectar posibles infracciones durante las fases en las que el jugador debe permanecer inmóvil.

Esta integración entre visión artificial y lógica de juego permite que las actividades sean más dinámicas e interactivas, ya que el robot es capaz de reaccionar a las acciones realizadas por el usuario sin necesidad de controles manuales adicionales. Además, el uso de una única fuente de datos visuales para todos los juegos reduce la complejidad del sistema y evita la duplicación de procesos de detección.

7. Verificación de las funcionalidades implementadas

7.1 Metodología de pruebas

Con el objetivo de verificar el correcto funcionamiento del sistema, se diseñó un conjunto de casos de prueba que permitieron validar las funcionalidades implementadas durante el desarrollo del proyecto.

Las pruebas se realizaron tanto en el entorno de simulación como sobre el robot real, evaluando de forma individual cada componente y comprobando posteriormente su integración con el resto del sistema. Para ello se utilizaron tres niveles de validación: pruebas unitarias, pruebas a nivel de nodo y pruebas integrales.

Cada caso de prueba incluye una descripción, un procedimiento de ejecución, el resultado esperado, el resultado obtenido y las evidencias correspondientes, permitiendo verificar de forma objetiva el cumplimiento de los requisitos definidos para el proyecto.



7.2 Lista de pruebas

Prueba	Tipo de prueba	Descripción	Resultado
H7 Juego Pilla pilla - P1	Integral	Inicio y ejecución del juego	Correcto. El juego se inicia correctamente y el robot comienza a navegar entre los puntos establecidos
H7 Juego Pilla pilla - P2	Integral	Finalización del juego	Correcto. El juego se detiene correctamente tras enviar la orden, cancelando la navegación
H10 Juego Escondite - P1	Integral	Inicio y ejecución del juego	Correcto. El juego se inicia correctamente y el robot comienza la búsqueda
H10 Juego Escondite - P2	Integral	Finalización del juego	Correcto. El juego se detiene correctamente al recibir la orden
H11 Página WEB - P1	A nivel de nodo	Conexión a robot desde la web exitosa	Correcto. La conexión se establece correctamente y el estado cambia a "Conectado"
H11 Página WEB - P2	Unitaria	Login correcto	Correcto. El usuario accede correctamente al sistema mostrando mensaje de éxito



H11 Página WEB - P3	Unitaria	Login incorrecto	Correcto. Se muestra mensaje de error indicando credenciales incorrectas
H11 Página WEB - P4	Unitaria	Cerrar sesión	Correcto. Se solicita confirmación y se cierra la sesión correctamente
H11 Página WEB - P5	A nivel de nodo	Comunicación entre la web y la API	Correcto. Las peticiones se realizan correctamente y la API responde con estado 200
H14 Navegación autónoma	Unitaria	Desplazamiento a un punto	Correcto. El robot navega hasta el punto indicado y permite cancelar la navegación

Tabla 46. Listado de las pruebas de validación funcional del sistema Patricio.

7.3 Resultados obtenidos

Los resultados obtenidos durante la fase de pruebas muestran que las funcionalidades principales del proyecto se ejecutan correctamente tanto en simulación como en el robot físico. Los casos de prueba realizados permitieron validar la navegación autónoma, la comunicación entre la interfaz web y ROS 2, la persistencia de datos, los sistemas de visión artificial y las actividades educativas implementadas.

Asimismo, las pruebas integrales confirmaron la correcta interacción entre los distintos componentes del sistema, verificando que la arquitectura desarrollada cumple los requisitos funcionales establecidos al inicio del proyecto.



8. Reflexión crítica del funcionamiento del grupo de trabajo y su evolución

La valoración general del trabajo en equipo durante el desarrollo de Patricio es positiva. A lo largo del proyecto hubo momentos en los que algunos integrantes tuvieron menos disponibilidad debido a responsabilidades externas, pero esto no afectó negativamente al resultado final, ya que cada uno supo organizarse para cumplir con sus tareas y responsabilidades dentro del equipo.

Como en cualquier proyecto de larga duración, también hubo fases en las que la motivación o la concentración no fueron siempre las mismas para todos los miembros. Sin embargo, el grupo supo adaptarse a estas situaciones y mantener el avance del proyecto mediante una comunicación constante y un reparto de trabajo flexible.

La resolución de problemas técnicos se realizó de forma colaborativa. Cuando algún integrante encontraba dificultades, el resto del equipo aportaba ideas o ayudaba a buscar soluciones, permitiendo superar los bloqueos con relativa rapidez. Esta dinámica fue especialmente importante durante las fases finales del proyecto, donde fue necesario integrar numerosos componentes desarrollados de forma paralela.

También hubo periodos de trabajo más intensos, especialmente cerca de determinadas entregas, en los que fue necesario dedicar horas adicionales para finalizar funcionalidades, corregir errores y preparar la documentación. Estas situaciones exigieron un esfuerzo extra por parte de todos los integrantes, pero contribuyeron a mantener el proyecto dentro de los plazos previstos.

A pesar de las dificultades encontradas durante el desarrollo, el ambiente de trabajo fue bueno durante todo el proyecto. Se mantuvo una relación basada en el respeto, la confianza y la colaboración, permitiendo que cada miembro pudiera aportar sus conocimientos y opiniones. Como resultado, el equipo consiguió completar satisfactoriamente los objetivos planteados y desarrollar una solución funcional que integra navegación autónoma, visión artificial, interacción por voz y actividades educativas.



Adenor Buret

Al principio nos costó un poco entendernos y encontrar una forma de trabajar que nos funcionara a todos. Hubo momentos en los que la coordinación no era la mejor y algunas tareas avanzaban más lento de lo esperado. Con el tiempo fuimos ajustándonos y las cosas empezaron a salir de forma más natural, sin necesidad de estar pendientes constantemente de quién hacía cada cosa. Personalmente, creo que una de las mejores noticias del proyecto es que, a pesar de la cantidad de cambios y pruebas que hicimos, no llegamos a romper nada importante. Al final conseguimos sacar el proyecto adelante y eso demuestra que, aunque no siempre lo hicimos perfecto, encontramos una forma de funcionar como equipo. t

Santiago Aguirre Crespo

Durante este sprint el trabajo del equipo ha avanzado de forma constante. Aunque algunas tareas han requerido más tiempo del previsto, en general la organización ha sido buena y cada miembro ha podido centrarse en sus responsabilidades.

También se han realizado varias pruebas de integración para comprobar que los distintos componentes funcionaban correctamente juntos. En algunos casos fue necesario realizar ajustes y correcciones, pero finalmente se consiguieron resolver los problemas encontrados. Considero que el ambiente de trabajo ha sido positivo y que la colaboración entre los miembros del grupo ha contribuido a que el proyecto siguiera avanzando de manera satisfactoria.

Pablo Meana

Durante este sprint el equipo ha seguido avanzando de forma satisfactoria en el desarrollo del proyecto. A medida que se han ido completando funcionalidades, también ha sido necesario dedicar tiempo a realizar pruebas, corregir errores y mejorar la integración entre los distintos módulos.

Considero que el trabajo realizado ha sido positivo y que cada miembro ha contribuido en las tareas que tenía asignadas. Además, cuando ha sido



necesario coordinar diferentes partes del proyecto, la comunicación ha sido buena y ha permitido resolver los problemas que han ido apareciendo. Aunque todavía quedaban algunos aspectos por finalizar, la evolución del proyecto durante este sprint ha sido favorable y ha permitido acercarnos a la versión final del sistema.

Mari Dapcheva

Creo que la organización del equipo al principio era bastante caótica, pero hemos mejorado mucho desde entonces. Con el paso de los sprints aprendimos a coordinarnos mejor y a repartir las tareas de forma más eficiente. Cuando surgían desacuerdos o diferentes opiniones, siempre se hablaban las cosas con respeto hasta llegar a una solución que convenciera a todos. También valoro que las ideas de cada miembro del equipo han sido escuchadas y tenidas en cuenta durante el desarrollo del proyecto. Personalmente, hubo momentos en los que no estuve centrada al cien por cien, pero en ningún momento me sentí juzgada por ello, algo que agradezco mucho al resto del equipo.

César Herrero

Durante este sprint el proyecto ha continuado avanzando de forma progresiva y se han ido completando varias de las tareas previstas. Además del desarrollo de nuevas funcionalidades, también se ha dedicado tiempo a realizar pruebas y corregir algunos errores detectados durante la integración de los distintos componentes.

En general, considero que el trabajo realizado ha sido positivo y que la organización del equipo ha permitido mantener un buen ritmo de desarrollo. La comunicación entre los miembros ha sido adecuada y ha facilitado la resolución de las dificultades que han surgido a lo largo del sprint. Estoy satisfecho con la evolución del proyecto y con los resultados obtenidos hasta el momento.

Juan Bautista Peris



Creo que, en general, nos hemos llevado bien durante todo el proyecto. Aunque en el primer sprint hubo algunos problemas de organización, poco a poco fuimos mejorando la forma de trabajar juntos. Con el paso de las semanas aprendimos a coordinarnos mejor y a repartir las tareas de manera más eficiente. Cuando surgían dudas o problemas técnicos, siempre había alguien dispuesto a ayudar. También supimos mantener un buen ambiente de trabajo incluso en los momentos de más presión. En general, considero que hemos funcionado como un equipo y que eso ha sido clave para completar el proyecto.

9. Conclusiones y propuestas de mejora

9.1 Conclusiones

El desarrollo de Patricio ha permitido diseñar e implementar un sistema robótico capaz de combinar navegación autónoma, visión artificial, interacción mediante voz y actividades educativas dentro de una misma plataforma. A lo largo del proyecto se han integrado tecnologías como ROS 2, Gazebo, OpenCV, MediaPipe, Flask y MySQL, consiguiendo una solución funcional tanto en simulación como en el robot físico.

Los resultados obtenidos durante las pruebas realizadas demuestran que las principales funcionalidades del sistema operan correctamente y cumplen los objetivos planteados al inicio del proyecto. Entre ellas destacan la navegación autónoma, la comunicación entre la interfaz web y ROS 2, la persistencia de datos, la detección corporal mediante visión artificial y la ejecución de los distintos juegos educativos desarrollados.

Además de los aspectos técnicos, el proyecto ha supuesto una oportunidad para adquirir experiencia en el desarrollo de sistemas distribuidos, la integración de múltiples tecnologías y el trabajo colaborativo dentro de un equipo multidisciplinar.



9.2 Propuestas de mejora

Aunque el sistema desarrollado cumple los objetivos establecidos, existen diversas líneas de mejora que podrían incorporarse en futuras versiones.

Una posible mejora consistiría en ampliar las capacidades conversacionales del robot mediante modelos de inteligencia artificial más avanzados, permitiendo mantener conversaciones más naturales y adaptadas a cada usuario.

También sería interesante incorporar nuevos juegos y actividades educativas personalizadas según la edad y las necesidades de cada niño, aumentando así el valor pedagógico de la plataforma.

Otra línea de trabajo consistiría en mejorar la relación entre el robot y las familias. Por ejemplo, podría desarrollarse una aplicación que permitiera a los padres supervisar las actividades realizadas por sus hijos, consultar estadísticas de uso o incluso interactuar con ellos a través del robot cuando no puedan estar físicamente presentes.

Finalmente, podría mejorarse el sistema de reconocimiento mediante nuevas técnicas de visión artificial capaces de identificar gestos, emociones o situaciones de riesgo con mayor precisión, ampliando las capacidades de interacción y asistencia del robot.

